

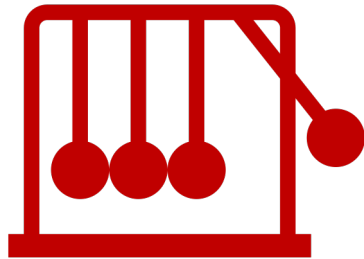
Coding challenge!

1. Choose a file from github: `shiny_nanocourse/data`
2. Start from the old faithful template shiny app
3. Quickly create an app that does the following:
 - Display the data in table format in tab #1. Include a short text description of the data.
 - Plot the data in tab #2. Use inputs in a sidebar to let users explore the data interactively.
 - In tab #3, calculate summary statistics for the plotted data
 - Make sure to use a custom theme to make it pretty. 😊

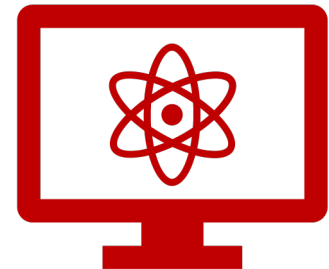
Interactive plots & files,

Nanocourse Lecture 4, 8/5/25

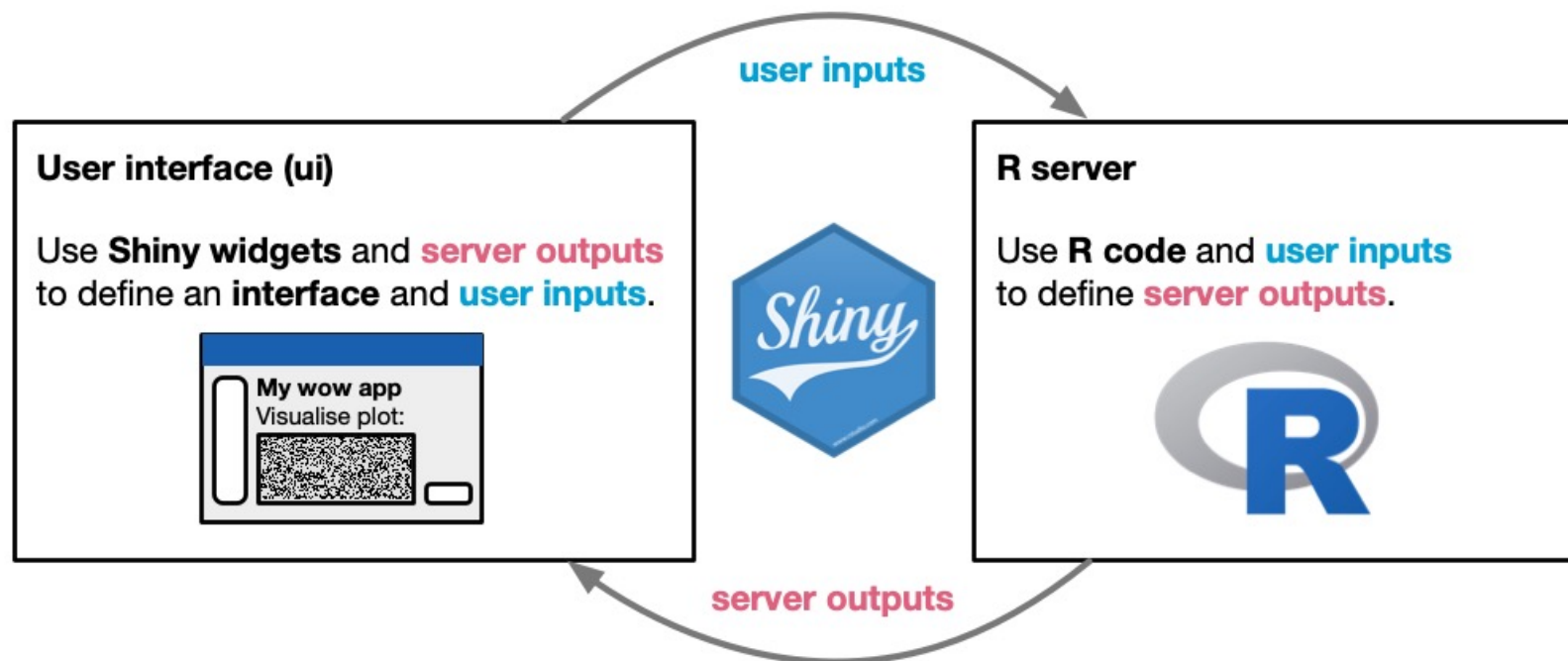
Scott Saunders



1010
1010



Using plot interactions as inputs



Using plot interactions as inputs

Examples

- [click](#)
- [brush](#)
- [linked plotlys](#)



Simple plot interactivity

```
# UI  
  
plotOutput(outputId = "distPlot")  
  
sliderInput("bins", min = 1, max = 50)
```



```
# Server  
  
output$distPlot <- renderPlot({  
  bins <- seq(min, max, input$bins)  
  hist(..., breaks = bins)  
})  
  
# Run app  
  
shinyApp(ui = ui, server = server)
```

Simple plot interactivity

```
# UI

plotOutput(outputId = "distPlot",
            click = "plot_click",
            hover = "plot_hover",
            brush = "plot_brush"
)

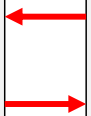
sliderInput("bins", min = 1, max = 50)
```

```
# Server

output$distPlot <- renderPlot({
  bins <- seq(min, max, input$bins)
  hist(..., breaks = bins)
})

# Run app

shinyApp(ui = ui, server = server)
```



Simple plot interactivity

```
# UI

plotOutput(outputId = "distPlot",
            hover = "plot_hover",
            ),

verbatimTextOutput("plot_info")

sliderInput("bins", min = 1, max = 50)
```

```
# Server

output$distPlot <- renderPlot({
  bins <- seq(min, max, input$bins)
  hist(..., breaks = bins)
})

output$plot_info <- renderPrint({
  input$plot_hover
})

# Run app

shinyApp(ui = ui, server = server)
```



Code exercise: Old faithful brushing

```
# UI

plotOutput(outputId = "distPlot",
           brush = "plot_brush",
           ),

verbatimTextOutput("plot_info")

sliderInput("bins", min = 1, max = 50)
```

```
# Server

output$distPlot <- renderPlot({
  ggplot(faithful,
        aes(x=waiting, y=eruptions)) +
    geom_point()
})

output$plot_info <- renderPrint({
  brushedPoints(
    faithful, input$plot_brush
  )
})

# Run app

shinyApp(ui = ui, server = server)
```


File upload / download

```
# UI
fileInput("file", accept = '.csv')

# Server

df <- reactive({

read_csv(input$file$datapath)

})
```



<https://mastering-shiny.org/action-transfer.html>

Code exercise: Curve fitting user data!

L4_logistic_files_app.R

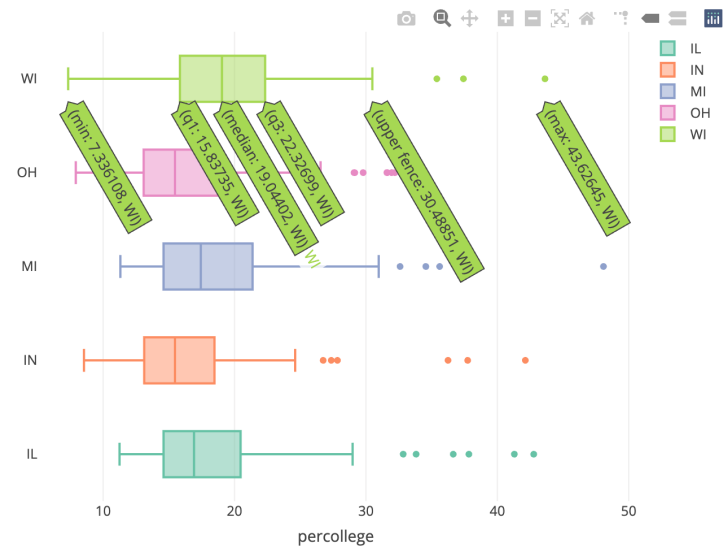


Plotly

```
# UI
plotlyOutput(outputId = "plot")

# Server
output$plot <- renderPlotly({})

  event_data("plotly_click")
```



<https://plotly-r.com/>

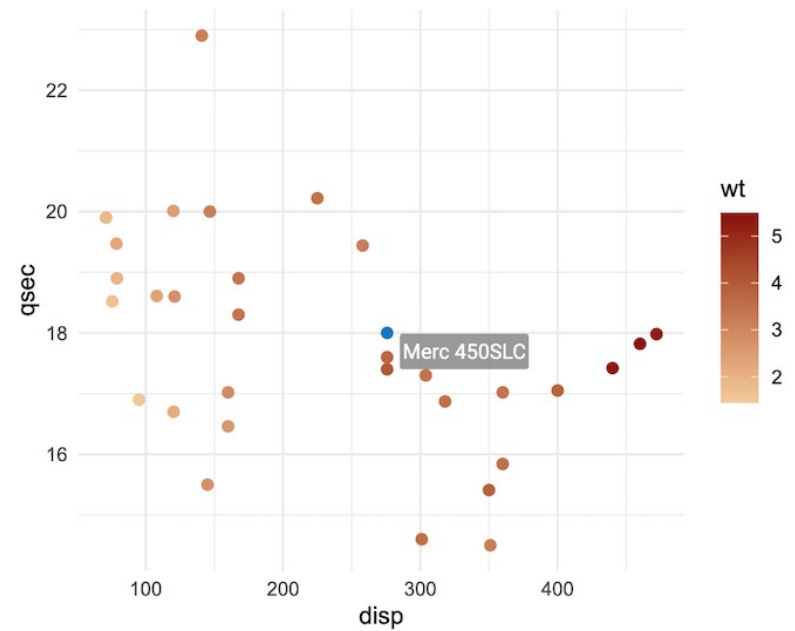
<https://davidgohel.github.io/ggiraph/>

Ggiraph

```
# UI
girafeOutput("plot")

# Server
output$plot <- renderGirafe({})

input$plot_selected
```



Code demo: Plotly & Ggiraph

L4_plotly_ggiraph_demo_app.R

